

Universidade do Minho
Licenciatura em Engenharia Informática

RELATÓRIO

SISTEMAS OPERATIVOS

TRABALHO PRÁTICO

ORQUESTRAÇÃO DE UM AMBIENTE MULTI-RUNNER

Trabalho realizado por:

Ana Patrícia da Silva Machado (*a111661*)
Carolina Francisca Vieira Dias (*a111179*)
Gonçalo Nuno Gomes Pombal (*a110681*)

Grupo 7

Maio, 2026

ÍNDICE

1. INTRODUÇÃO....1
1.1 Descrição do Problema....1
1.2 Objetivos do Trabalho....1
2. ARQUITETURA DO SISTEMA....1
2.1 Visão Geral....1
Figura 1 - Arquitetura do sistema de orquestração multi-runner....2
2.2 Arquitetura de Processos....2
3. MECANISMOS DE COMUNICAÇÃO....2
3.1 Escolha e Justificação dos Named Pipes....2
Figura 2 - Protocolo de comunicação entre runner, controller e processo filho....3
3.2 Estrutura Message/Response....3
3.3 FIFO Global do Controller....4
3.4 FIFOs Privados do Runner....4
4. IMPLEMENTAÇÃO DO RUNNER....4
4.1 Parsing de Argumentos....4
4.2 Modo -e: Execução de Comandos....4
4.3 Modo -c: Consulta de Estado....5
4.4 Modo -s: Shutdown....5
4.5 Execução de Comandos e Suporte a Operadores....5
Figura 3 - Execução de comandos com os operadores |, >, < e 2> através do runner....5
5. IMPLEMENTAÇÃO DO CONTROLLER....6
5.1 Receção de Mensagens....6
5.2 Política de Escalonamento....6
5.3 Gestão de Comandos em Execução e em Espera....6
5.4 Registo Persistente em ficheiro....6
5.5 Shutdown Controlado....7
6. POLÍTICA DE ESCALONAMENTO....7
6.1 Descrição das Políticas Implementadas....7
6.2 Justificação das Escolhas....7
6.3 Comparação entre Políticas....8
7. AVALIAÇÃO EXPERIMENTAL....8
7.1 Descrição dos Testes....8
7.2 Paralelismo....8
7.3 Resultados....8
Tabela 1 - Duração total por utilizador nas quatro configurações testadas (ms)....9
7.4 Análise e Conclusões....10
8. CONCLUSÃO....10

1. INTRODUÇÃO

1.1 Descrição do Problema

O presente relatório descreve o desenvolvimento de um sistema de orquestração de execução de comandos bash, realizado no âmbito da unidade curricular de Sistemas Operativos da Universidade do Minho. O sistema é composto por dois programas em C, o *runner* e o *controller*, que comunicam exclusivamente através de named pipes (FIFOs), aplicando as primitivas lecionadas na UC: `fork()`, `execvp()`, `pipe()` e `dup2()`. O *runner* permite submeter comandos, consultar o estado do sistema e solicitar a sua terminação; o *controller* é responsável por escalonar e gerir a execução concorrente desses comandos.

1.2 Objetivos do Trabalho

Os principais objetivos são a implementação correta dos dois programas, o suporte a operadores de redirecionamento (`>`, `2>`, `<`) e encadeamento via `|`, e a avaliação experimental de diferentes políticas de escalonamento e configurações de paralelismo.

2. ARQUITETURA DO SISTEMA

2.1 Visão Geral

O sistema segue um modelo cliente-servidor assimétrico: o *runner* funciona como cliente e o *controller* como servidor. Existe um FIFO global (`/tmp/controller_fifo`) através do qual todos os runners enviam os seus pedidos. Para receber respostas, cada runner cria o seu próprio FIFO privado (`/tmp/runner_fifo_<pid>`), identificado pelo PID, garantindo que as respostas são entregues ao runner correto mesmo com múltiplos runners em execução simultânea.

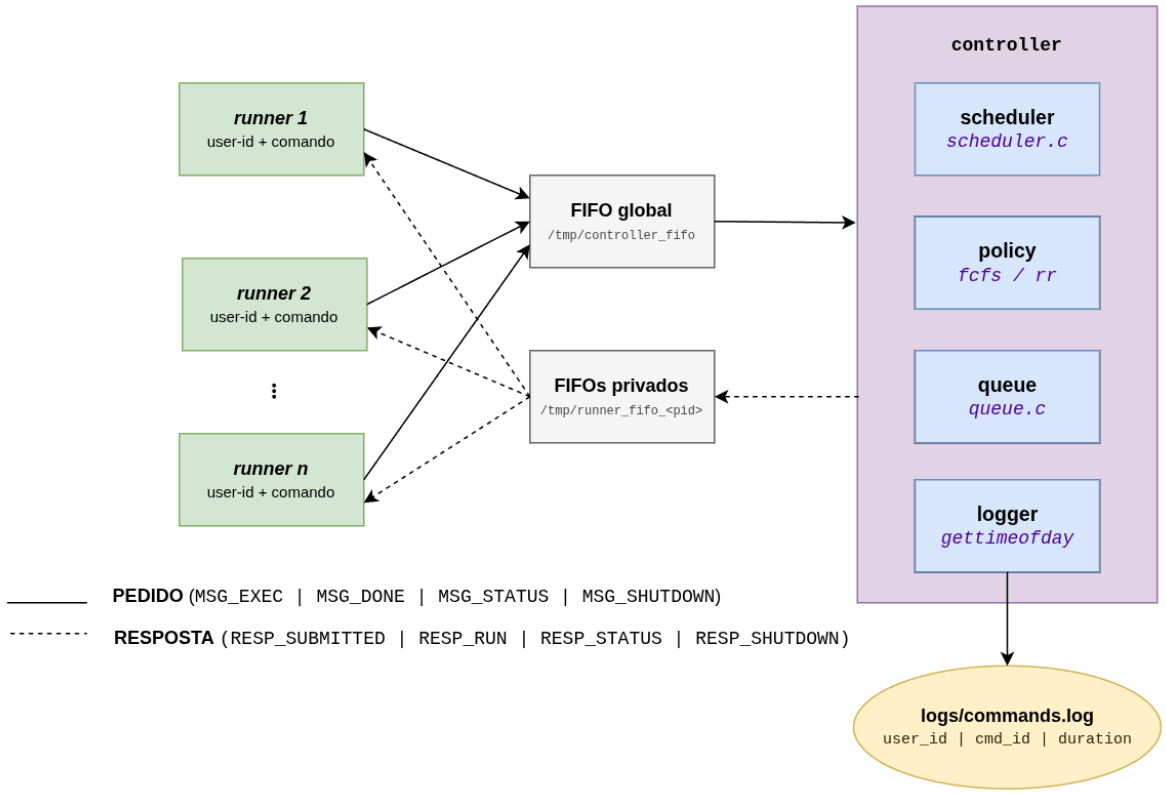


***Figura 1** - Arquitetura do sistema de orquestração multi-runner*

2.2 Arquitetura de Processos

O *controller* corre como um único processo que recebe e processa mensagens dos runners de forma concorrente, recorrendo aos módulos `scheduler.c`, `policy.c` e `queue.c` para gerir o escalonamento. Para cada comando autorizado, o *runner* cria um processo filho com `fork()` que executa o comando via `execvp()`. O processo pai aguarda com `waitpid()` e notifica o controller com `MSG_DONE` após a conclusão. A compilação segue a Makefile do enunciado, com os targets all, controller, runner e clean.

3. MECANISMOS DE COMUNICAÇÃO

3.1 Escolha e Justificação dos Named Pipes

A comunicação é realizada através de named pipes (FIFOs) porque os dois programas são processos independentes sem relação de parentesco, o que impossibilita o uso de pipes anónimos. Os named pipes permitem comunicação entre processos distintos através do sistema de ficheiros, seguindo uma semântica FIFO. O protocolo completo de troca de mensagens é ilustrado na Figura 2.

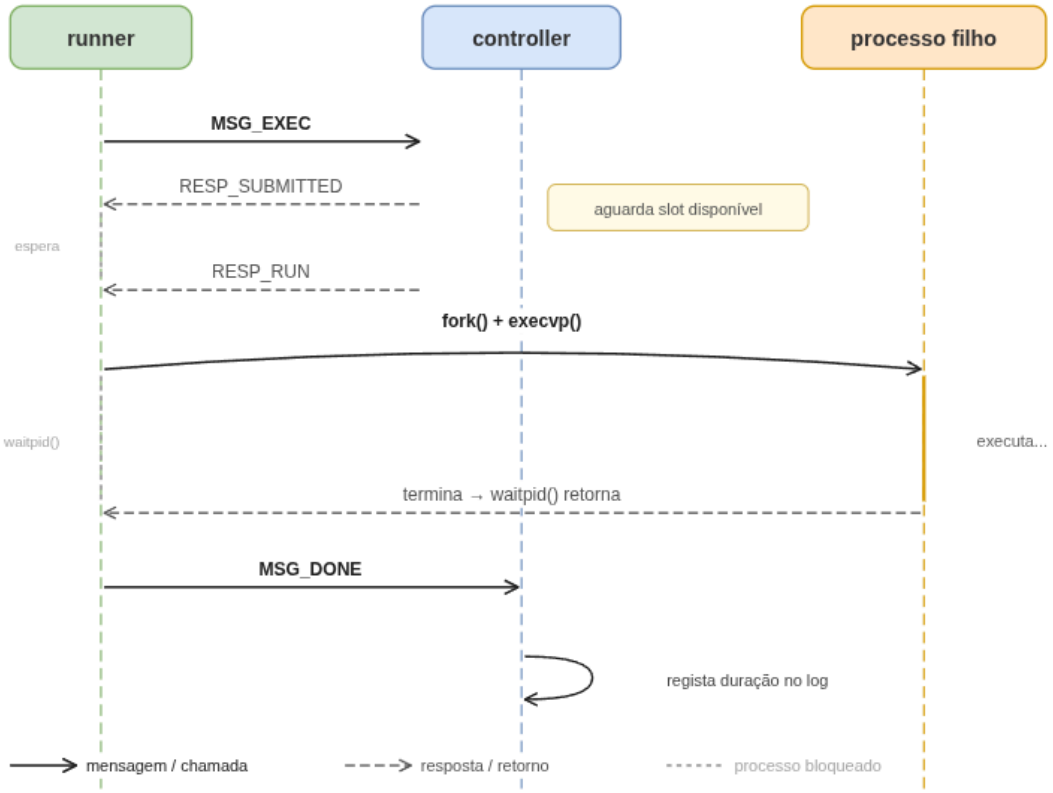


***Figura 2** - Protocolo de comunicação entre runner, controller e processo filho*

3.2 Estrutura Message/Response

Toda a comunicação é feita através de duas estruturas definidas em `utils.h`: `Message` (Runner → Controller) e `Response` (Controller → Runner).

A estrutura `Message` contém o campo `type` que identifica a operação (`MSG_EXEC`, `MSG_DONE`, `MSG_STATUS`, `MSG_SHUTDOWN`) e o campo `runner_pid`, usado pelo controller para identificar o FIFO privado de destino da resposta:

```
typedef struct {
    MsgType type;
    pid_t   runner_pid;
    int     user_id;
    int     cmd_id;
    char    cmd[MAX_CMD_LEN];
} Message;
```

A estrutura `Response` controla o fluxo no runner através do campo type: `RESP_SUBMITTED` confirma a entrada na fila de espera; `RESP_RUN` autoriza o início da execução com fork(); `RESP_STATUS` transporta a lista de comandos no campo body; `RESP_SHUTSUBM` confirma ao runner que pediu o shutdown que o pedido foi aceite; `RESP_DENIED` sinaliza que o pedido foi recusado por o controller já estar em processo de terminação; `RESP_SHUTDOWN` confirma a terminação efetiva do controller e é enviada ao runner que pediu o shutdown após todos os comandos pendentes terem concluído.

3.3 FIFO Global do Controller

O controller disponibiliza um FIFO global em `/tmp/controller_fifo` com permissões `0666`, criado no arranque e removido com `unlink()` na terminação. Este constitui o único ponto de entrada do sistema, onde todos os runners, independentemente do utilizador ou operação, enviam as suas mensagens por este canal.

3.4 FIFOs Privados do Runner

Cada runner cria o seu FIFO privado em `/tmp/runner_fifo_<pid>` no início de cada operação e remove-o com `unlink()` no final. Este mecanismo garante que as respostas do controller são entregues exclusivamente ao runner correto, mesmo em cenários de alta concorrência.

4. IMPLEMENTAÇÃO DO RUNNER

4.1 Parsing de Argumentos

O *runner* aceita três modos definidos pelo primeiro argumento: `-e` (executar), `-c` (consultar estado) e `-s` (shutdown). A identificação é feita em `main()` com `strcmp()`, redirecionando para a função correspondente.

4.2 Modo `-e`: Execução de Comandos

O modo `-e` é o modo principal. O runner cria o seu FIFO privado, constrói uma `MSG_EXEC` com `user_id`, `cmd_id` (igual ao PID do runner, garantindo unicidade) e cmd, e envia-a ao controller pelo FIFO global. O protocolo de espera é em dois passos: o runner aguarda `RESP_SUBMITTED` para notificar o utilizador da submissão e, em seguida, `RESP_RUN` para iniciar a execução, garantindo que esta não começa prematuramente. Após `RESP_RUN`, o runner executa o comando, envia `MSG_DONE` ao controller e notifica o utilizador da conclusão.

4.3 Modo `-c`: Consulta de Estado

O runner envia `MSG_STATUS` pelo FIFO global e aguarda uma `RESP_STATUS` no FIFO privado. O campo body, preenchido pelo controller com a lista de comandos em execução e em espera por ordem de despacho da política ativa (`cmd_id` ascendente no FCFS; ordem do cursor `rr_cursor` no Round-Robin), é apresentado diretamente no stdout.

4.4 Modo `-s`: Shutdown

O runner envia `MSG_SHUTDOWN` ao controller e aguarda confirmação no FIFO privado. Após receber `RESP_SHUTDOWN`, notifica o utilizador que o controller terminou.

4.5 Execução de Comandos e Suporte a Operadores

A execução é realizada diretamente com `fork()` e `execvp()`, sem recurso ao interpretador de comandos. O comando em `argv[3]`, que pode conter operadores `|`, `>`, `2>` e `<` , é processado em dois módulos independentes: `parser.c` e `executor.c`.

A função `tokenize()` divide a string por espaços e passa o resultado a `parse_pipeline()`, que constrói uma estrutura Pipeline com um ou mais Command, onde cada `|` inicia um novo Command, e os operadores `>`, `2>` e `<` são consumidos com o token seguinte para os campos output, error e input. A execução é delegada a `execute_pipeline()`, que cria n-1 pipes anónimos para uma pipeline de n comandos; cada processo filho aplica os redirecionamentos com `dup2()` antes de invocar `execvp()`, e o processo pai fecha todos os pipes e aguarda todos os filhos com `waitpid()`, garantindo que `MSG_DONE` só é enviado ao controller após a execução estar completa. A Figura 3 ilustra os quatro operadores suportados.



Figura 3 - *Execução de comandos com os operadores |, >, < e 2> através do runner*

5. IMPLEMENTAÇÃO DO CONTROLLER

5.1 Receção de Mensagens

O controller abre o FIFO global com `O_RDWR | O_NONBLOCK`. O flag `O_RDWR` é necessário para evitar que `open()` bloqueie à espera de um escritor, em que o processo é simultaneamente leitor e escritor do descritor, pelo que o FIFO abre imediatamente. O flag `O_NONBLOCK` faz com que `read()` devolva `EAGAIN` quando não há dados disponíveis, permitindo que o loop verifique outras condições, como o shutdown, entre leituras. Quando `read()` devolve `EAGAIN`, o controller aguarda 10ms com `usleep()` antes de tentar novamente, evitando busy waiting.

5.2 Política de Escalonamento

O controller delega toda a lógica de escalonamento ao módulo scheduler.c, chamando scheduler_enqueue() ao receber MSG_EXEC e scheduler_next() para obter o próximo comando a despachar. A política ativa é definida no arranque via argumento e passada a scheduler_init(). O controller não conhece os detalhes da política, apenas recebe um CmdNode de scheduler_next() e abre o FIFO privado do runner correspondente para enviar RESP_RUN.

5.3 Gestão de Comandos em Execução e em Espera

Ao receber MSG_EXEC, o controller enfileira o comando com scheduler_enqueue(), notifica imediatamente o runner com RESP_SUBMITTED (para que este possa mostrar a mensagem de submissão ao utilizador), e tenta 'despachar' com scheduler_next(). Se houver slot livre, o runner recebe RESP_RUN de seguida; caso contrário, permanece bloqueado no FIFO privado até que um slot fique disponível. Quando o runner notifica a conclusão com MSG_DONE, o controller chama scheduler_done() e volta a chamar scheduler_next() para despachar o próximo comando em espera.

5.4 Registo Persistente em ficheiro

Após receber MSG_DONE, o controller abre logs/commands.log com O_WRONLY | O_CREAT | O_APPEND e regista uma linha com o user_id, cmd_id e duração em milissegundos. A duração é calculada pelo scheduler_done() com gettimeofday(), medindo o tempo desde a entrada na fila até à conclusão do comando. O uso de O_APPEND garante que a linha é registada a partir do último offset da escrita anterior, impedindo que escritas concorrentes corrompam o ficheiro.

5.5 Shutdown Controlado

Ao receber MSG_SHUTDOWN, o controller responde com RESP_SHUTSUBM e marca shutdown_requested = 1. Novos pedidos MSG_EXEC e MSG_SHUTDOWN são recusados com RESP_DENIED, mas os comandos já submetidos continuam a ser despachados em cada MSG_DONE através de scheduler_next(), drenando progressivamente a fila. O ciclo principal termina quando scheduler_empty() devolve verdadeiro (sem comandos em execução nem em espera), momento em que o controller envia RESP_SHUTDOWN ao runner requisitante e remove o FIFO global.

6. POLÍTICA DE ESCALONAMENTO

6.1 Descrição das Políticas Implementadas

Foram implementadas duas políticas, selecionáveis no arranque do controller:

- **FCFS (First-Come, First-Served):** implementada em pick_fcfs(), percorre as filas de todos os utilizadores e devolve o CmdNode com o cmd_id mais baixo globalmente, ou seja, o mais antigo. A ordem de chegada é preservada independentemente do utilizador.

- **Round-Robin por utilizador:** implementada em pick_rr(), mantém um cursor *rr_cursor* que aponta para o último utilizador servido. A cada chamada, começa pelo utilizador seguinte ao cursor e percorre a lista circularmente até encontrar um CMD_WAITING. O cursor é validado a cada chamada para garantir que não aponta para um utilizador entretanto removido da lista.

Os utilizadores são geridos em queue.c numa lista ligada de UserQueue, cada uma com a sua própria fila de CmdNode. Quando a fila de um utilizador fica vazia após scheduler_done(), a UserQueue é libertada com free_user_if_empty().

6.2 Justificação das Escolhas

O FCFS foi escolhido como política base por ser simples de implementar, fácil de raciocinar e suficiente para o caso base do enunciado. O Round-Robin foi escolhido como segunda política por garantir equidade entre utilizadores, nenhum utilizador pode monopolizar o sistema enquanto outros têm comandos em espera.

A estrutura de dados escolhida foi uma lista ligada de listas ligadas (uma por utilizador) com alocação dinâmica. Esta abordagem elimina o limite fixo de entradas na fila, adapta-se naturalmente ao Round-Robin por utilizador, e liberta memória quando um utilizador não tem mais comandos.

6.3 Comparação entre Políticas

Como pode ser observado nos testes (secção 7), com parallel-commands=1 o FCFS apresenta tempo total ligeiramente inferior ao Round-Robin, porque serve os comandos pela ordem global de chegada sem introduzir alternância. O Round-Robin neste cenário penaliza utilizadores com comandos mais rápidos ao forçar a alternância entre utilizadores.

Com parallel-commands=2, o Round-Robin demonstra maior equidade ao alternar entre utilizadores no preenchimento dos slots disponíveis, distribui o tempo de espera de forma mais equilibrada. O FCFS com paralelismo tende a favorecer os utilizadores que submeteram mais cedo, podendo atrasar utilizadores que chegaram ligeiramente depois mesmo que os seus comandos sejam mais curtos.

7. AVALIAÇÃO EXPERIMENTAL

7.1 Descrição dos Testes

Os testes avaliam o comportamento do sistema sob diferentes configurações de escalonamento e paralelismo. Para cada configuração, foram lançados 4 runners em paralelo com comandos de duração diferente (`sleep 1`, `sleep 2`, `sleep 3`, `sleep 3`). A métrica principal é a duração total de cada comando desde a submissão até à conclusão, registada em `logs/commands.log`. Os testes foram automatizados com `tests/test_scheduling.sh`.

Foram testadas quatro configurações:

- **FCFS, paralelo 1:** execução sequencial por ordem de chegada
- **RR, paralelo 1:** execução sequencial com alternância entre utilizadores
- **FCFS, paralelo 2:** até 2 comandos simultâneos por ordem de chegada
- **RR, paralelo 2:** até 2 comandos simultâneos com alternância entre utilizadores

7.2 Paralelismo

O controller suporta execução paralela configurável via argumento `parallel-commands`. Com `parallel-commands=1` o sistema é sequencial. Com `parallel-commands=2`, dois comandos executam em simultâneo, reduzindo significativamente os tempos de espera, onde dois `sleep 10` lançados em paralelo terminam ambos em ~10 s em vez dos ~20 s expectáveis numa execução sequencial.

7.3 Resultados

A tabela seguinte apresenta as durações registadas pelo controller para cada utilizador nas quatro configurações testadas, incluindo o tempo de espera em fila.

Tabela 1 - *Duração total por utilizador nas quatro configurações testadas (ms)*

Configuração	user 1 (sleep 3)	user 2 (sleep 1)	user 3 (sleep 2)	user 4 (sleep 3)	Tempo total
FCFS, paralelo 1	3002 ms	4005 ms	6007 ms	9009 ms	9009 ms
RR, paralelo 1	3002 ms	6005 ms	8008 ms	9011 ms	9011 ms
FCFS, paralelo 2	3001 ms	1001 ms	3003 ms	6003 ms	6003 ms
RR, paralelo 2	3002 ms	1001 ms	4003 ms	5004 ms	5004 ms

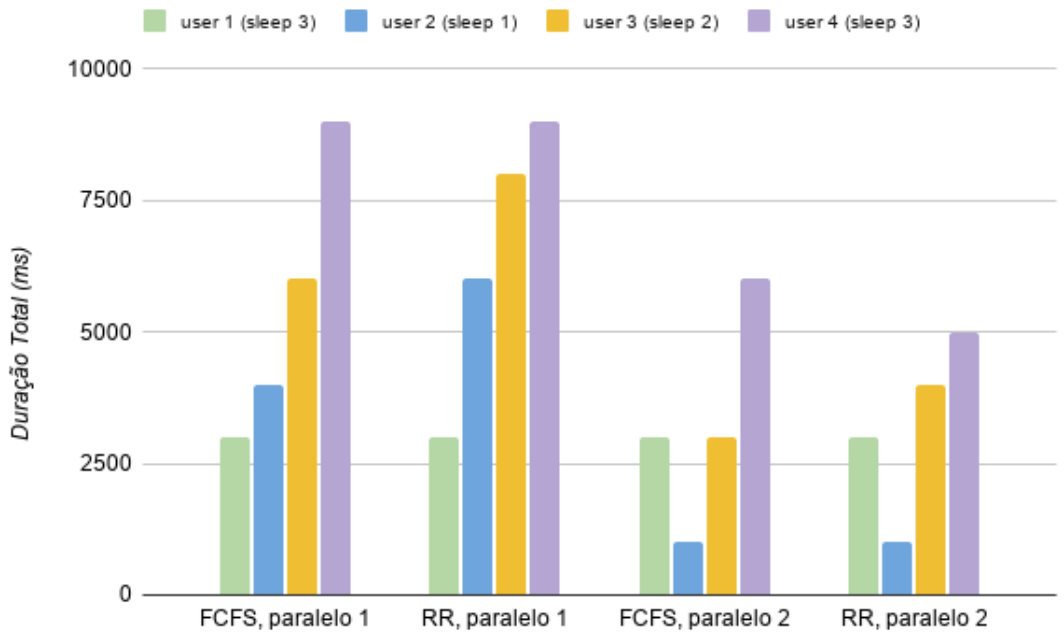


Figura 4 *- Duração total por utilizador nas quatro configurações testadas (ms)*

7.4 Análise e Conclusões

Impacto do paralelismo: a variável com maior impacto no desempenho global é o grau de paralelismo. Passar de `parallel-commands=1` para `parallel-commands=2` reduziu o tempo total de ~9 s para ~6 s no FCFS (redução de 33%) e de ~9 s para ~5 s no Round Robin (redução de 44%), uma vez que os dois primeiros comandos da fila executam imediatamente em paralelo.

FCFS vs Round Robin com paralelismo 1: com um único slot, o FCFS apresentou tempo total ligeiramente inferior. O Round Robin atrasa utilizadores subsequentes ao garantir que todos têm uma execução antes de repetir, penalizando os utilizadores 2, 3 e 4 neste cenário específico.

FCFS vs Round Robin com paralelismo 2: com dois slots disponíveis, o Round Robin demonstrou maior equidade — o utilizador 4 esperou apenas 5004 ms em vez dos 6003 ms registados com FCFS. Com paralelismo suficiente, a alternância entre utilizadores traduz-se numa distribuição mais equilibrada dos tempos de espera.

➔ **Conclusão:** o aumento do paralelismo é o fator com maior impacto no desempenho global. O Round Robin revela-se mais justo que o FCFS em cenários com paralelismo, garantindo que nenhum utilizador é sistematicamente preterido.

8. CONCLUSÃO

O trabalho realizado permitiu implementar com sucesso um sistema de orquestração de execução de comandos bash, cumprindo todos os objetivos propostos. A comunicação via named pipes, a execução concorrente com paralelismo configurável, o suporte a pipelines e operadores de redirecionamento, as políticas FCFS e Round Robin, e o registo persistente de comandos foram todos implementados e validados experimentalmente.

Ao longo do desenvolvimento surgiram algumas dificuldades, sobretudo relacionadas com a sincronização entre processos. Garantir que o runner não iniciava a execução prematuramente exigiu um protocolo de dois passos entre `RESP_SUBMITTED` e `RESP_RUN`, e a sincronização do shutdown, assegurando que o controller só termina após todos os comandos em execução concluírem, revelou-se a parte mais delicada da implementação.

Como trabalho futuro, seria interessante adicionar suporte a prioridades por utilizador, um mecanismo de timeout para comandos demasiado longos e persistência do estado da fila entre reinícios do controller.